Applied Big Data Engineering
Mini Project Assessment

Department of Computer Engineering
Faculty of Engineering
University of Ruhuna Sri Lanka

Assignment by

Dissanayaka D.M.S.C.	-	EG/2020/3905
Bandara U.G.R.S.	-	EG/2020/3847
Dissanayake D.M.M.I.T.	-	EG/2020/3912

Scenario 1: The "Smart City" Traffic & Congestion System

1. Introduction

Modern urbanization has caused a rapid increase in traffic congestion. It poses challenges for city planners and commuters. Colombo as a developing metropolitan city needs a sophisticated traffic management system to address these issues. This report presents an overview of the architecture and implementation of an end-to-end data pipeline to monitor, analyze and report on traffic conditions in real-time. The system uses a Lambda architecture to deliver real-time insights for tactical reaction and historical analysis for strategic purposes. The project is aligned with the requirements of the “Smart City” Traffic & Congestion System scenario and uses the latest big data stack including Apache Kafka, Apache Spark and Apache Airflow.

2. Justification of Technology Stack

The selection of technologies was guided by the project's dual requirements for real-time stream processing and reliable batch analysis. The chosen stack represents a robust, scalable, and widely adopted solution for big data challenges.

- **Architecture - Lambda Architecture**

The Lambda architecture was chosen because it provides a fault-tolerant and comprehensive data processing solution. The reason for choosing the Lambda architecture is that it provides a complete and fault tolerant way of processing data. It breaks the system into three layers called Speed layer, Batch Layer and Serving Layer. The Speed Layer for real-time, low-latency data processing. The Batch Layer is used for high accuracy historical analysis, The Serving Layer is used to serve views from both layers. This separation works well for our case too, where we want to trigger immediate alerts (speed drops below 10 km/h), but also want to do complex resource heavy calculations on a whole day of data (find peak traffic hours). However, a Kappa architecture, which only uses a stream processing engine, could simplify the system. The Lambda approach is more robust, as the batch layer can reprocess historical data to fix any mistakes

- **Data Ingestion - Apache Kafka**

Apache Kafka is the de facto standard for building real-time data pipelines. Its suitability for this project stems from several key features like high throughput and scalability, Durability and Fault Tolerance, Decoupling of Producers and Consumers.

High Throughput and Scalability	Kafka can handle millions of messages per second, making it capable of ingesting high-velocity data from thousands of IoT sensors simultaneously. As Colombo expands its sensor network, Kafka can scale horizontally to meet the demand.
Durability and Fault Tolerance	Data is persisted to disk and replicated across the cluster, ensuring that no sensor readings are lost even if a broker or consumer fails.
Decoupling of Producers and Consumers	Kafka acts as a central buffer, allowing the data producers (sensors) and consumers (Spark, databases) to operate independently at their own pace. This prevents the stream processing engine from being overwhelmed by data spikes.

- **Stream and Batch Processing - Apache Spark**

Apache Spark was selected as the unified processing engine for both the speed and batch layers.

Unified API	Spark's Structured Streaming API provides a high-level, declarative syntax for stream processing that is nearly identical to its batch processing API (DataFrames). This significantly reduces development complexity, as the same logic can be adapted for both real-time and historical analysis.
Performance	Spark's in-memory processing capabilities deliver high performance for both streaming and batch workloads.
Rich Ecosystem	It integrates seamlessly with Kafka for data ingestion and with various storage systems like Parquet and PostgreSQL. Its support for windowing functions is critical for calculating the Congestion Index over 5-minute intervals as required.

- **Orchestration - Apache Airflow**

For the batch layer, a reliable orchestration tool is needed to schedule and manage the nightly data aggregation jobs. Apache Airflow is the ideal choice due to its programmability, monitoring and Management and extensibility.

Programmability	DAGs (Directed Acyclic Graphs) are defined in Python, which allows complex, dynamic workflows. This made it easy to define the sequence of tasks: run the Spark batch job, generate the visualization and finally print a summary.
Monitoring and Management	Airflow has a rich UI to monitor job status, view logs and trigger runs manually. This is an essential feature for managing and debugging the nightly reporting pipeline.
Extensibility	It has a huge library of operators and hooks making it easy to integrate almost any system including Spark and PostgreSQL.

- **Storage - PostgreSQL and Parquet**

To meet the different needs we decided to go with a hybrid storage strategy.

For a serving layer we used a relational database, PostgreSQL. It is fantastic for storing structured queryable data such as the real time critical traffic alerts and the final peak traffic stats report. The transactional nature of it ensures data integrity. It is also easily accessible for downstream applications or dashboards.

Parquet was the best option for the historical data lake. It is a columnar storage format that offers very good compression and query performance, especially for the analytical workloads that the Spark batch job runs. But you store the original, immutable event data in Parquet on a distributed file system, and you have a scalable, cost-effective "single source of truth."

3. Handling of Event Time vs. Processing Time

In a real-time data pipeline, it is important to distinguish between Event Time (time when an event actually happened) and Processing Time (time when the system processes the event) for correct analysis. Data can arrive late or out of order due to latency in the network or downtime of the system.

This project tackles this problem with the native features of Spark Structured Streaming. How those features are used is explained below.

Embedding Event Time in Data	The Python producer script embeds a timestamp in each JSON message indicating the time of the sensor reading generation. This is Event Time.
Watermarking for Late Data	The function withWatermark("timestamp", "10 minutes") is used in the spark_processor.py script for this. A watermark tells the Spark engine to wait for a certain amount of time (10 minutes) for late data to show up before the engine closes out the calculation of a window. For example, if a window closes at 5:05 PM, the engine won't process that window until its internal clock reaches 5:15 PM. Any data that arrives for that window after the watermark has passed is considered to be late, and it is dropped. This mechanism provides a trade-off between completeness and latency, so that the system will not wait forever for late events, but will still allow for reasonable delays.
Windowing on Event Time	The aggregation groupBy(window(col("timestamp"), "5 minutes"), ...) is being performed on the timestamp column (the Event Time) and not on the time the data was processed. In this way, the 'Congestion Index' is computed based on the real time when the traffic conditions occurred at the junction instead of the processing delays.

By implementing watermarking and windowing on the event timestamp, the pipeline guarantees that the analytical results, such as the Congestion Index and Peak Traffic Hour, accurately reflect the real-world traffic patterns as they happened.

4. Ethics and Data Governance

Smart city traffic management systems can be a good thing, but they raise serious ethical questions around surveillance and data privacy. The data collected seems anonymous but could be used to infer personal travel patterns.

- **Privacy Implications and Surveillance**

The continuous gathering of vehicle counts and speeds from fixed sensors can result in a kind of mass surveillance. This project does not use license plate recognition, but in a future extension such a system could be used. Tracking a vehicle's movement across the city, identifying an individual's daily commute, home location and places of work or worship could be achieved by combining data from multiple sensors. This information is highly sensitive. If the dataset were to be breached or misused, it could expose individuals to risks such as stalking, burglary (by knowing when they are not home), or profiling based on their travel habits.

- **Data Governance and Mitigation Strategies**

To tackle these ethical issues, a robust data governance framework is essential. The following principles should be followed.

1. Data Anonymization and Aggregation - The main mitigation strategy is the use of aggregated and anonymized data wherever possible. The current system implementation collects only vehicle counts and average speeds, not individual vehicle identifiers. Any future improvements should be guided by this principle. Data should be stored and analysed in aggregate form (5-minute totals) so that no individuals can be re-identified.

2. Purpose Limitation - Data collected should solely be used for the explicit purpose of traffic management. But there should be a clear policy against its use for law enforcement (to issue speeding tickets) or commercial purposes (targeted advertising) without public consent and legal justification.

3. Access Control - Implement strict access controls to ensure that only authorized personnel have access to the raw and processed data. This includes role-based access, encrypting data at rest and in transit, and regular security audits.

4. Data Retention Policies - The data retention policy should specify the duration for which data is stored. For example, raw sensor data might be retained for 30 days for batch analysis, and then deleted or aggregated into coarser, less sensitive historical summaries.

5. Transparency - The public should be aware of what data is being collected, why it is being collected and how it is to be used. This transparency builds public trust and enables oversight.

By embedding these ethical considerations and data governance practices into the system's design and operation, the city of Colombo can reap the benefits of a smart traffic management system while respecting and protecting the privacy of its citizens.